Secure AI Prompt Sandbox

Technical Threat report

Sat Mar 07 2026 20:43:50 GMT+0000 (Coordinated Universal Time)

Description:
Web-based LLM gateway security middleware. Sits between users and LLM APIs, enforcing prompt validation, injection detection, RBAC, context isolation, and tamper-evident audit logging.

Filtered by: Critical risk | High risk | Unmitigated threats

This report only includes threats that match the filters above.

Workflow status: Draft

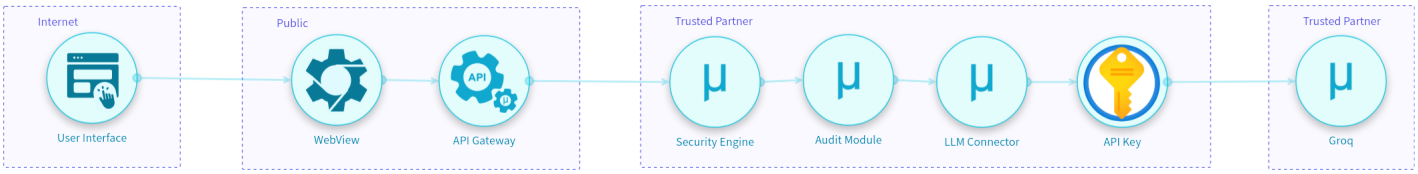
Project tags: Web Application

Unique ID: secure-ai-prompt-sandbox-1772883962536

Owner: deez homie

Updated: Mar 7, 2026, 8:41 PM

Project number: secure-ai-prompt-sandbox-1772883962536



Content menu

Risk Mitigation Summary

Unmitigated Threats

Partly Mitigated Threats

Mitigated Threats

Accepted Threats

- API Gateway
- API Key
- Audit Module
- Groq
- LLM Connector
- Security Engine
- User Interface
- WebView

Not-Applicable Threats

Failed Countermeasures

Appendix A: Threat Details

- API Gateway
- API Key
- Audit Module
- Groq
- LLM Connector
- Security Engine
- User Interface
- WebView

Risk Mitigation Summary

Risk summary

Current risk level compared with the inherent and projected risk levels.

Inherent Risk:	60%
Current Risk:	60%
Projected Risk:	0%

Risk distribution

Number of threats in each risk rating level.

Very Low:	0
Low:	0
Medium:	5
High:	29
Critical:	4

Unmitigated : 0 | Partly-Mitigated : 0 | Mitigated : 0 | Accepted : 38 | N/A : 12

Vulnerabilities: 48 | **Total number of threats:** 38

Countermeasures state

Implemented:	0
Required:	57
Recommended:	0

Rejected : 0 | N/A : 6

Failed	0	Verified	0

Unmitigated Threats

Listed below are threats per component where all countermeasures are not implemented or the weaknesses test result failed.

No Data

Partly Mitigated Threats

Listed below are threats per component where some countermeasures are not implemented or the weaknesses test result failed.

No Data

Mitigated Threats

Below is the list of threats per component where all countermeasures are implemented and have passed their tests and there are no failed weakness tests.

No Data

Accepted Threats

List of accepted threats per component, with the corresponding reason for acceptance.

Component: API Gateway

Threats of this component

Use case: Elevation of Privilege

AT1. Threat name: Authentication bypass

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Repudiation

AT2. Threat name: Exploitation of insufficient logging and monitoring

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Component: API Key

Threats of this component

Use case: Elevation of Privilege

AT3. Threat name: Unauthorized access through misconfigured Azure RBAC role assignments

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Information Disclosure

AT4. Threat name: Unauthorized access to Azure Key Vault leading to data compromise

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT5. Threat name: Avoid embedding credentials in code or configuration files, use secure storage

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT6. Threat name: Insecure management of key lifecycle in Azure Key Vault

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT7. Threat name: Unauthorized access due to misconfigured Azure Key Vault settings

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT8. Threat name: Unauthorized access to Azure Key Vault via public network

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Component: Audit Module

Threats of this component

Use case: Information Disclosure

AT9. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data

- **Reason the risk is accepted:**

The threats were going to work on in our SSD project

AT10. Threat name: Sensitive data is compromised through network access

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT11. Threat name: Sensitive data is exposed through weak security configurations

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Tampering

AT12. Threat name: Attackers compromise images by modifying their content

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Component: Groq

Threats of this component

Use case: Information Disclosure

AT13. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT14. Threat name: Sensitive data is compromised through network access

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT15. Threat name: Sensitive data is exposed through weak security configurations

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Component: LLM Connector

Threats of this component

Use case: Information Disclosure

AT16. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT17. Threat name: Sensitive data is compromised through network access

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT18. Threat name: Sensitive data is exposed through weak security configurations

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Tampering

AT19. Threat name: Attackers compromise images by modifying their content

- **Reason the risk is accepted:**

The threats were going to work on in our SSD project

Component: Security Engine

Threats of this component

Use case: Denial of Service

AT20. Threat name: Availability is compromised through attacks against scalability configuration

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Information Disclosure

AT21. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT22. Threat name: Sensitive data is compromised through network access

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT23. Threat name: Sensitive data is exposed through weak security configurations

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Tampering

AT24. Threat name: Attackers compromise images by modifying their content

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Component: User Interface

Threats of this component

Use case: Denial of Service

AT25. Threat name: An attacker can perform denial-of-service (DoS) attacks on the user interface

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Spoofing

AT26. Threat name: An attacker can perform clickjacking attacks

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT27. Threat name: An attacker can perform UI redressing attacks

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Tampering

AT28. Threat name: An attacker can perform cross-site scripting (XSS) attacks

- **Reason the risk is accepted:**

The threats were going to work on in our SSD project

Component: WebView

Threats of this component

Use case: Elevation of Privilege

AT29. Threat name: WebView injection attacks

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Information Disclosure

AT30. Threat name: Man-in-the-middle (MitM) attack on WebView

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT31. Threat name: Sensitive data is exposed through WebView

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

AT32. Threat name: Untrusted content loading in WebView

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Use case: Tampering

AT33. Threat name: Cross-site scripting (XSS) via WebView

- **Reason the risk is accepted:**
The threats were going to work on in our SSD project

Not-Applicable Threats

List of not-applicable threats per component, with the corresponding reason for not-applicability.

No Data

Failed Countermeasure Tests

No Data

Appendix A: Threat Details

The number of threats in each risk rating level.

Component: API Gateway

Threats of this component in each risk rating level

AT1. Threat name: Authentication bypass [T-API-GATEWAY-T-API-GW-01]

- **State:** Accepted
- **Description:** **General threat description**
  An attack scenario in which malicious actors find ways to circumvent the authentication mechanisms, leading to unapproved access to API functionalities.
  Threat agents/Attack vectors
  Attackers may exploit vulnerabilities in the authentication process or use stolen credentials to gain access without proper authorization.
  Impacts
  Such actions can result in unauthorized data access, data breaches, or service misuse, compromising both security and privacy.
  Example Attack Scenarios
  An attacker could intercept unencrypted credentials or leverage a brute-force attack to bypass login controls, gaining entry to sensitive API endpoints without detection.
- **C0. Countermeasure name:** Connectors should be provided for integrating with identity providers (IdPs)
  - **State:** REQUIRED
- **C1. Countermeasure name:** The API gateway should have a connector to an artifact that can generate an access token for the client request
  - **State:** REQUIRED
- **C2. Countermeasure name:** Integrate the API gateway with an identity management application
  - **State:** REQUIRED
- **C3. Countermeasure name:** Distributed gateway deployments should have a token translation (exchange) service between gateways
  - **State:** REQUIRED

AT2. Threat name: Exploitation of insufficient logging and monitoring [T-API-GATEWAY-T-API-GW-02]

- **State:** Accepted
- **Description:**

General threat description
The absence of adequate logging and monitoring allows attackers to execute malicious actions without detection. This can lead to unauthorized access and potential data breaches with the exploitation remaining hidden from defenders.
Threat agents/Attack vectors
Cybercriminals may exploit this weakness by injecting malicious code into the API gateway or by initiating a series of unauthorized requests. Without proper monitoring, these activities can go unnoticed.
Impacts
This threat can lead to compromised systems, unauthorized data access, and potential data loss or manipulation. Moreover, it may result in delayed response to breaches and increased recovery costs.
Example Attack Scenarios
An attacker successfully bypasses authentication controls and gains access to sensitive data through the API gateway. Due to the absence of logging, the attacker maintains access over an extended period, exfiltrating data without triggering alerts.

- **C4. Countermeasure name:** Securely channel all traffic information to a monitoring and/or analytics application
  - **State:** REQUIRED

Component: API Key

AT3. Threat name: Unauthorized access through misconfigured Azure RBAC role assignments [T-KEY-VAULT-7]

- **State:** Accepted
- **Description:**

General threat description
Unauthorized access to Azure resources can occur if Azure RBAC roles are not properly managed and assigned, leading to potential data breaches, service disruptions, or unauthorized actions within the Azure environment.
Threat agents/Attack vectors
Threat agents could include malicious insiders, external attackers, or unauthorized users gaining access to Azure resources through misconfigured or overly permissive Azure RBAC role assignments. Attack vectors may include exploiting weak authentication mechanisms, leveraging stolen credentials, or abusing overly permissive role assignments.
Impacts
The impacts of this threat include unauthorized access to sensitive data, unauthorized modification or deletion of resources, service disruptions, or unauthorized use of Azure services, leading to potential data breaches, financial loss, reputational damage, and compliance violations.
Example Attack Scenarios
An attacker gains access to an Azure subscription through compromised credentials and exploits overly permissive Azure RBAC role assignments to escalate privileges, access sensitive data, and modify critical resources without proper authorization. Alternatively, a malicious insider exploits their legitimate access to improperly assign themselves overly permissive Azure RBAC roles, allowing them to perform unauthorized actions within the Azure environment.

- **C5. Countermeasure name:** Follow just enough administration (least privilege) principle
  - **State:** REQUIRED

AT4. Threat name: Unauthorized access to Azure Key Vault leading to data compromise [T-KEY-VAULT-5]

- **State:** Accepted
- **Description:**

General threat description
Enabling Microsoft Defender for Key Vault provides protection and detection capabilities for key management services, helping to safeguard against unauthorized access, data exfiltration, and other security threats. In the event of an alert, investigating and responding promptly is crucial to mitigate potential security risks.
Threat agents/Attack vectors
Threat agents may attempt to gain unauthorized access to sensitive keys and secrets stored in the Key Vault, either through exploiting vulnerabilities in the Key Vault or through unauthorized

access to the surrounding infrastructure. Attack vectors may include credential theft, exploitation of misconfigured access controls, and insider threats.
Impacts
The impact of a successful attack on the Key Vault could result in unauthorized access to sensitive keys and secrets, potential data exfiltration, and compromise of cryptographic materials. This could lead to data breaches, loss of sensitive information, and potential impact on the confidentiality, integrity, and availability of critical systems and data.
Example Attack Scenarios
An attacker gains access to administrative credentials through phishing or credential theft and exfiltrates sensitive keys from the Key Vault, enabling them to decrypt and access sensitive data. Another scenario involves an insider threat misusing their authorized access to the Key Vault to retrieve and exfiltrate sensitive keys for malicious purposes.

- **C6. Countermeasure name:** Enable threat detection capabilities
  - **State:** REQUIRED
- **C7. Countermeasure name:** Enable network logging for security investigation
  - **State:** REQUIRED

AT5. Threat name: Avoid embedding credentials in code or configuration files, use secure storage [T-KEY-VAULT-4]

- **State:** Accepted
- **Description:**

General threat description
Storing secrets and credentials in code or configuration files can lead to unauthorized access and potential exposure of sensitive information, which can pose a significant security risk.
Threat agents/Attack vectors
Attackers with access to the codebase or configuration files, insider threats, or unauthorized individuals who gain access to the application or infrastructure can exploit hardcoded credentials and secrets.
Impacts
The impacts of storing secrets and credentials in code or configuration files include potential unauthorized access to sensitive data, service or data tampering, data breaches, and loss of trust from customers and stakeholders.
Example Attack Scenarios
An attacker gains access to the source code repository and locates hardcoded credentials, allowing them to compromise the associated Azure resources, access sensitive data, or tamper with the application's functionality. Alternatively, an unauthorized individual gains access to the configuration files on a compromised system, leading to unauthorized access to Azure resources or sensitive data.

- **C8. Countermeasure name:** Restrict the exposure of credential and secrets
  - **State:** REQUIRED
- **C9. Countermeasure name:** Restrict resource access based on conditions
  - **State:** REQUIRED
- **C10. Countermeasure name:** Manage application identities securely and automatically
  - **State:** REQUIRED

AT6. Threat name: Insecure management of key lifecycle in Azure Key Vault [T-KEY-VAULT-3]

- **State:** Accepted
- **Description:**

General threat description
The threat of unauthorized access and compromise of certificates and keys stored in Azure Key Vault due to misconfigurations, weak security controls, or exploitation of vulnerabilities.
Threat agents/Attack vectors
Threat agents could include malicious insiders, external attackers, or individuals with unauthorized access to the Azure Key Vault. Attack vectors may involve exploiting weak access controls, leveraging stolen credentials, or abusing vulnerabilities in the key vault infrastructure.
Impacts
The impacts of a successful attack could result in unauthorized access to sensitive certificates and keys, leading to data breaches, unauthorized decryption of data, and potential compromise of sensitive systems and information.
Example Attack Scenarios
1. An attacker exploits misconfigured access policies in Azure Key Vault, gaining unauthorized access to certificates and keys, allowing them to decrypt sensitive data or impersonate trusted entities. 2. A malicious insider abuses their privileges to improperly rotate or delete keys and certificates, leading to service disruption or data loss. 3. Vulnerabilities in the key vault infrastructure are exploited to extract sensitive key and certificate information, potentially leading to further attacks or data compromise.

- **C11. Countermeasure name:** Use a secure key management process
  - **State:** REQUIRED
- **C12. Countermeasure name:** Use a secure certificate management process
  - **State:** REQUIRED
- **C13. Countermeasure name:** Use customer-managed key option in data at rest encryption when required
  - **State:** REQUIRED

AT7. Threat name: Unauthorized access due to misconfigured Azure Key Vault settings [T-KEY-VAULT-1]

- **State:** Accepted
- **Description:**

General threat description
Misconfigured Azure Key Vault settings can expose sensitive data to unauthorized access, leading to potential data breaches and leakage of critical information. Attackers may exploit misconfigurations to gain access to encryption keys, secrets, and certificates stored within the Key Vault, compromising the overall security of the cloud environment.
Threat agents/Attack vectors
Threat agents may include malicious actors leveraging automated tools to scan for misconfigured Azure Key Vaults or insiders with access to the Azure environment seeking to exploit inadequate security configurations. Attack vectors may involve brute force attacks, credential theft, and unauthorized access to Key Vault resources directly or through other compromised Azure resources.
Impacts
The impacts of misconfigured Azure Key Vaults may result in unauthorized access to sensitive data, encryption keys, and credentials. This could lead to data exfiltration, unauthorized decryption of sensitive information, and exploitation of compromised credentials, ultimately leading to reputational damage, financial loss, and non-compliance with data protection regulations.
Example Attack Scenarios
An attacker identifies a misconfigured Azure Key Vault with weak access control policies and gains unauthorized access to encryption keys and secrets. Leveraging the accessed credentials, the attacker exfiltrates sensitive data or decrypts confidential information, leading to data breaches and compliance violations.

- **C14. Countermeasure name:** Use only approved services
  - **State:** REQUIRED

AT8. Threat name: Unauthorized access to Azure Key Vault via public network [T-KEY-VAULT-6]

- **State:** Accepted

- **Description:**

General threat description
An attacker with malicious intent may attempt to gain unauthorized access to Azure Key Vault or intercept sensitive data by exploiting the lack of proper network controls and access restrictions in place.
Threat agents/Attack vectors
Unauthenticated users, malicious insiders, or external attackers may attempt to bypass the network security controls or exploit misconfigured access rules to gain access to the Azure Key Vault resources.
Impacts
The impact of a successful attack could result in data breaches, theft of sensitive information, unauthorized access to encryption keys, or compromise of critical secrets stored in the Azure Key Vault.
Example Attack Scenarios
An attacker may attempt to exploit the lack of proper IP filtering rules in the Azure Key Vault firewall to gain unauthorized access from public networks. Additionally, an attacker may try to circumvent the network security groups (NSGs) to gain access to open ports and services that are meant to be restricted to trusted networks. Finally, an attacker may attempt to intercept traffic or perform man-in-the-middle attacks on communication with the Key Vault by exploiting the absence of private endpoints or virtual network service endpoints.

- **C15. Countermeasure name:** Secure cloud native services with network controls
  - **State:** REQUIRED
- **C16. Countermeasure name:** Establish network segmentation boundaries
  - **State:** REQUIRED

Component: Audit Module

AT10. Threat name: Sensitive data is compromised through network access [T-MICROSERVICE-T-MICROSERVICE-03]

- **State:** Accepted
- **Description:** **General threat description**
Inadequate protection of network traffic can lead to unauthorized access to sensitive information, such as credentials or personal data, during transmission.
Threat agents/Attack vectors
Attackers might exploit weak or absent encryption protocols by intercepting data transmitted between services or between a client and a service.
Impacts
Compromise of confidential data could lead to data breaches, loss of customer trust, and regulatory penalties.
Example Attack Scenarios
An attacker positions themselves on the network where they can eavesdrop on unencrypted API requests, capturing user login credentials.
- **C17. Countermeasure name:** Use secure network communication protocols
  - **State:** REQUIRED

AT11. Threat name: Sensitive data is exposed through weak security configurations [T-MICROSERVICE-T-MICROSERVICE-06]

- **State:** Accepted
- **Description:**

General threat description
Improper security configurations in software components can result in the unintended exposure of sensitive data to unauthorized entities, potentially allowing them to access or manipulate confidential information.
Threat agents/Attack vectors
Authorized users, malicious insiders, or external attackers who exploit misconfigured security settings to gain unauthorized access or uncover sensitive information.
Impacts
Unauthorized data access can lead to data breaches, loss of customer trust, legal ramifications, and financial loss for the organization.
Example Attack Scenarios
An attacker may exploit a security misconfiguration in a microservice that fails to enforce strong encryption, thereby intercepting sensitive data in transit. Alternatively, a poorly configured API endpoint could expose sensitive customer data to unauthorized users.

- **C18. Countermeasure name:** Implement secure configurations and data protection measures
  - **State:** REQUIRED

AT12. Threat name: Attackers compromise images by modifying their content [T-MICROSERVICE-T-MICROSERVICE-04]

- **State:** Accepted
- **Description:**

General threat description
Unauthorized modifications to container images can introduce vulnerabilities, allowing potential attackers to compromise the integrity and security of the software components.
Threat agents/Attack vectors
Attackers may exploit misconfigured access controls or leverage supply chain weaknesses to alter container images without authorization.
Impacts
Such unauthorized modifications could lead to the deployment of insecure services, data breaches, and potential service outages.
Example Attack Scenarios
An attacker gains access to a container registry due to weak credentials and replaces a legitimate image with a malicious version, leading to the execution of harmful code when deployed.

- **C19. Countermeasure name:** Secure container images and reduce attack surface
  - **State:** REQUIRED

AT9. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data [T-MICROSERVICE-T-MICROSERVICE-07]

- **State:** Accepted
- **Description:**

General threat description
Inadequate configuration settings and weaknesses in cloud setups can lead to exposure of sensitive data to unauthorized individuals or entities.
Threat agents/Attack vectors
Malicious actors could exploit cloud service misconfigurations or vulnerabilities to gain unauthorized access to data stored within the cloud infrastructure.
Impacts
Unauthorized access to sensitive data can lead to data breaches, loss of confidentiality, financial loss, and reputational damage to the organization.
Example Attack Scenarios
An attacker leverages an exposed API due to improper configuration to access confidential data, or an oversight in access permissions allows a threat actor to extract and exploit sensitive information from cloud storage.

- **C20. Countermeasure name:** Enforce strict access control and minimize credential exposure
  - **State:** REQUIRED

Component: Groq

AT13. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data [T-MICROSERVICE-T-MICROSERVICE-07]

- **State:** Accepted
- **Description:**

General threat description
Inadequate configuration settings and weaknesses in cloud setups can lead to exposure of sensitive data to unauthorized individuals or entities.
Threat agents/Attack vectors
Malicious actors could exploit cloud service misconfigurations or vulnerabilities to gain unauthorized access to data stored within the cloud infrastructure.
Impacts
Unauthorized access to sensitive data can lead to data breaches, loss of confidentiality, financial loss, and reputational damage to the organization.
Example Attack Scenarios
An attacker leverages an exposed API due to improper configuration to access confidential data, or an oversight in access permissions allows a threat actor to extract and exploit sensitive information from cloud storage.

- **C21. Countermeasure name:** Enforce strict access control and minimize credential exposure
  - **State:** REQUIRED

AT14. Threat name: Sensitive data is compromised through network access [T-MICROSERVICE-T-MICROSERVICE-03]

- **State:** Accepted
- **Description:** **General threat description**
  Inadequate protection of network traffic can lead to unauthorized access to sensitive information, such as credentials or personal data, during transmission.
  Threat agents/Attack vectors
  Attackers might exploit weak or absent encryption protocols by intercepting data transmitted between services or between a client and a service.
  Impacts
  Compromise of confidential data could lead to data breaches, loss of customer trust, and regulatory penalties.
  Example Attack Scenarios
  An attacker positions themselves on the network where they can eavesdrop on unencrypted API requests, capturing user login credentials.

- **C22. Countermeasure name:** Use secure network communication protocols
  - **State:** REQUIRED

AT15. Threat name: Sensitive data is exposed through weak security configurations [T-MICROSERVICE-T-MICROSERVICE-06]

- **State:** Accepted
- **Description:**

General threat description
Improper security configurations in software components can result in the unintended exposure of sensitive data to unauthorized entities, potentially allowing them to access or manipulate confidential information.
Threat agents/Attack vectors
Authorized users, malicious insiders, or external attackers who exploit misconfigured security settings to gain unauthorized access or uncover sensitive information.
Impacts
Unauthorized data access can lead to data breaches, loss of customer trust, legal ramifications, and financial loss for the organization.
Example Attack Scenarios
An attacker may exploit a security misconfiguration in a microservice that fails to enforce strong encryption, thereby intercepting sensitive data in transit. Alternatively, a poorly configured API endpoint could expose sensitive customer data to unauthorized users.

- **C23. Countermeasure name:** Implement secure configurations and data protection measures
  - **State:** REQUIRED

Component: LLM Connector

AT16. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data [T-MICROSERVICE-T-MICROSERVICE-07]

- **State:** Accepted
- **Description:**

General threat description
Inadequate configuration settings and weaknesses in cloud setups can lead to exposure of sensitive data to unauthorized individuals or entities.
Threat agents/Attack vectors
Malicious actors could exploit cloud service misconfigurations or vulnerabilities to gain unauthorized access to data stored within the cloud infrastructure.
Impacts
Unauthorized access to sensitive data can lead to data breaches, loss of confidentiality, financial loss, and reputational damage to the organization.
Example Attack Scenarios
An attacker leverages an exposed API due to improper configuration to access confidential data, or an oversight in access permissions allows a threat actor to extract and exploit sensitive information from cloud storage.

- **C24. Countermeasure name:** Enforce strict access control and minimize credential exposure
  - **State:** REQUIRED

AT17. Threat name: Sensitive data is compromised through network access [T-MICROSERVICE-T-MICROSERVICE-03]

- **State:** Accepted
- **Description:** **General threat description**
  Inadequate protection of network traffic can lead to unauthorized access to sensitive information, such as credentials or personal data, during transmission.
  Threat agents/Attack vectors

Attackers might exploit weak or absent encryption protocols by intercepting data transmitted between services or between a client and a service.
Impacts
Compromise of confidential data could lead to data breaches, loss of customer trust, and regulatory penalties.
Example Attack Scenarios
An attacker positions themselves on the network where they can eavesdrop on unencrypted API requests, capturing user login credentials.

- **C25. Countermeasure name:** Use secure network communication protocols
  - **State:** REQUIRED

AT18. Threat name: Sensitive data is exposed through weak security configurations [T-MICROSERVICE-T-MICROSERVICE-06]

- **State:** Accepted
- **Description:**

General threat description
Improper security configurations in software components can result in the unintended exposure of sensitive data to unauthorized entities, potentially allowing them to access or manipulate confidential information.
Threat agents/Attack vectors
Authorized users, malicious insiders, or external attackers who exploit misconfigured security settings to gain unauthorized access or uncover sensitive information.
Impacts
Unauthorized data access can lead to data breaches, loss of customer trust, legal ramifications, and financial loss for the organization.
Example Attack Scenarios
An attacker may exploit a security misconfiguration in a microservice that fails to enforce strong encryption, thereby intercepting sensitive data in transit. Alternatively, a poorly configured API endpoint could expose sensitive customer data to unauthorized users.

- **C26. Countermeasure name:** Implement secure configurations and data protection measures
  - **State:** REQUIRED

AT19. Threat name: Attackers compromise images by modifying their content [T-MICROSERVICE-T-MICROSERVICE-04]

- **State:** Accepted
- **Description:**

General threat description
Unauthorized modifications to container images can introduce vulnerabilities, allowing potential attackers to compromise the integrity and security of the software components.
Threat agents/Attack vectors
Attackers may exploit misconfigured access controls or leverage supply chain weaknesses to alter container images without authorization.
Impacts
Such unauthorized modifications could lead to the deployment of insecure services, data breaches, and potential service outages.
Example Attack Scenarios
An attacker gains access to a container registry due to weak credentials and replaces a legitimate image with a malicious version, leading to the execution of harmful code when deployed.

- **C27. Countermeasure name:** Secure container images and reduce attack surface
  - **State:** REQUIRED

Component: Security Engine

AT20. Threat name: Availability is compromised through attacks against scalability configuration [T-MICROSERVICE-T-MICROSERVICE-02]

- **State:** Accepted
- **Description:**

General threat description
The vulnerability lies in the scalability configuration of the microservices, where improper configuration or lack of resources can lead to decreased service availability.
Threat agents/Attack vectors
Malicious actors or automated tools can exploit misconfigurations by initiating large numbers of requests, overloading the service and causing denial of service.
Impacts
The primary impact is on the availability of the service, which can lead to downtime and potential loss of revenue, as well as damage to user trust.
Example Attack Scenarios
An attacker might exploit a weakness in the configuration by sending a burst of requests that exceed the service's capacity, leading to slower response times or complete service outages.

- **C28. Countermeasure name:** Ensure resilient scalability configurations
  - **State:** REQUIRED

AT21. Threat name: Attackers use known cloud vulnerabilities to access unauthorized data [T-MICROSERVICE-T-MICROSERVICE-07]

- **State:** Accepted
- **Description:**

General threat description
Inadequate configuration settings and weaknesses in cloud setups can lead to exposure of sensitive data to unauthorized individuals or entities.
Threat agents/Attack vectors
Malicious actors could exploit cloud service misconfigurations or vulnerabilities to gain unauthorized access to data stored within the cloud infrastructure.
Impacts
Unauthorized access to sensitive data can lead to data breaches, loss of confidentiality, financial loss, and reputational damage to the organization.
Example Attack Scenarios
An attacker leverages an exposed API due to improper configuration to access confidential data, or an oversight in access permissions allows a threat actor to extract and exploit sensitive information from cloud storage.

- **C29. Countermeasure name:** Enforce strict access control and minimize credential exposure
  - **State:** REQUIRED

AT22. Threat name: Sensitive data is compromised through network access [T-MICROSERVICE-T-MICROSERVICE-03]

- **State:** Accepted
- **Description:** **General threat description**
  Inadequate protection of network traffic can lead to unauthorized access to sensitive information, such as credentials or personal data, during transmission.
  Threat agents/Attack vectors
  Attackers might exploit weak or absent encryption protocols by intercepting data transmitted between services or between a client and a service.

Impacts
Compromise of confidential data could lead to data breaches, loss of customer trust, and regulatory penalties.
Example Attack Scenarios
An attacker positions themselves on the network where they can eavesdrop on unencrypted API requests, capturing user login credentials.

- **C30. Countermeasure name:** Use secure network communication protocols
  - **State:** REQUIRED

AT23. Threat name: Sensitive data is exposed through weak security configurations [T-MICROSERVICE-T-MICROSERVICE-06]

- **State:** Accepted
- **Description:**

General threat description
Improper security configurations in software components can result in the unintended exposure of sensitive data to unauthorized entities, potentially allowing them to access or manipulate confidential information.
Threat agents/Attack vectors
Authorized users, malicious insiders, or external attackers who exploit misconfigured security settings to gain unauthorized access or uncover sensitive information.
Impacts
Unauthorized data access can lead to data breaches, loss of customer trust, legal ramifications, and financial loss for the organization.
Example Attack Scenarios
An attacker may exploit a security misconfiguration in a microservice that fails to enforce strong encryption, thereby intercepting sensitive data in transit. Alternatively, a poorly configured API endpoint could expose sensitive customer data to unauthorized users.

- **C31. Countermeasure name:** Implement secure configurations and data protection measures
  - **State:** REQUIRED

AT24. Threat name: Attackers compromise images by modifying their content [T-MICROSERVICE-T-MICROSERVICE-04]

- **State:** Accepted
- **Description:**

General threat description
Unauthorized modifications to container images can introduce vulnerabilities, allowing potential attackers to compromise the integrity and security of the software components.
Threat agents/Attack vectors
Attackers may exploit misconfigured access controls or leverage supply chain weaknesses to alter container images without authorization.
Impacts
Such unauthorized modifications could lead to the deployment of insecure services, data breaches, and potential service outages.
Example Attack Scenarios
An attacker gains access to a container registry due to weak credentials and replaces a legitimate image with a malicious version, leading to the execution of harmful code when deployed.

- **C32. Countermeasure name:** Secure container images and reduce attack surface
  - **State:** REQUIRED

Component: User Interface

AT25. Threat name: An attacker can perform denial-of-service (DoS) attacks on the user interface [T-USER-INTERFACE-DOS-ATTACK]

- **State:** Accepted
- **Description:**

General Threat Description:
A Denial-of-Service (DoS) attack on the User Interface (UI) targets the elements that users interact with to disrupt service and render the application unusable. Unlike traditional network-level DoS attacks that saturate the server with traffic, UI-focused DoS attacks aim to overload or break the interface components, thereby preventing legitimate user actions and interactions. These attacks may involve flooding the UI with excessive requests, manipulating UI components to cause failures, or exploiting vulnerabilities that cause the application to hang or crash.
Threat Agents/Attack Vectors:
The primary agents of UI DoS attacks are often external attackers who may not require authenticated access to the application. Attack vectors can include:
Automated scripts that repeatedly trigger heavy UI processes.
Exploitation of flaws in client-side scripts like JavaScript to initiate intensive operations that freeze the UI.
Abusive use of legitimate functions, such as overly complex search queries or rapid sequence actions that the UI isn't designed to handle efficiently.
Impacts:
The impacts of a UI DoS attack are significant and can include:
Degradation of application performance leading to slow response times or total unavailability.
Reduced productivity and potential revenue loss if users cannot complete transactions or access services.
Erosion of trust in the application's reliability, which might deter users from returning.
Example Attack Scenarios:
Scenario 1: An attacker develops a script that automates the process of opening and closing an advanced options menu in the application's UI rapidly and continuously. This menu requires significant processing power to render. The script triggers this function faster than the application can handle, leading to a buffer overflow error that crashes the application for all users.
Scenario 2: An e-commerce website's product search feature is exploited where the attacker sends complex, nested search queries at a high frequency. Each query is designed to be computationally expensive, invoking deep database searches and extensive result rendering efforts. The server remains operational, but the UI becomes irresponsive to legitimate user queries due to the load, effectively denying service.

- **C33. Countermeasure name:** Use load balancing and scaling
  - **State:** REQUIRED
- **C34. Countermeasure name:** Implement rate limiting
  - **State:** REQUIRED

AT26. Threat name: An attacker can perform clickjacking attacks [T-USER-INTERFACE-CLICKJACKING-ATTACK]

- **State:** Accepted
- **Description:**

General Threat Description:
Clickjacking, also known as a "UI redress attack," involves tricking a user into clicking on something different from what the user perceives, effectively hijacking the user's intended actions. This attack leverages embedded components, such as an invisible or disguised iframe, overlaid on legitimate UI elements. The attacker manipulates visibility and positioning to coax the user into performing actions like clicking on a button or link that triggers unintended operations. These operations could range from granting permissions and accessing camera or microphone to liking social media content or initiating financial transactions.
Threat Agents/Attack Vectors:
Clickjacking primarily exploits vulnerabilities in the way web applications handle UI layering and user input. Threat agents include:

Malicious website operators who embed the target application within an iframe on their site with deceptive overlays.
Adversaries distributing links through emails, social media, or other platforms that lead to such malicious pages.

Impacts:
The impacts of clickjacking can vary widely based on the nature of the unintended action but typically include:
Unauthorized actions performed on behalf of the user, leading to data theft, account compromise, or financial loss.
Compromised privacy, if permissions are manipulated to access hardware like cameras or microphones.
Erosion of user trust in the security of the application.

Example Attack Scenarios:
Scenario 1: A user visits a seemingly benign news article shared on social media. Unknown to the user, the page contains an invisible iframe overlaying a popular social networking site where the user is still logged in. The article prompts the user to click to "reveal" content, but the click actually submits a "change password" request on the social network, with the new password controlled by the attacker.
Scenario 2: An attacker sets up a fake game website where users are encouraged to click on various parts of the screen to interact with the game. However, each click is actually on an invisible overlay that causes the user to unknowingly purchase items on an e-commerce platform, using a session the user had previously authenticated but not logged out of.

- **C35. Countermeasure name:** Use x-frame-options header
  - **State:** REQUIRED
- **C36. Countermeasure name:** Implement frame busting techniques
  - **State:** REQUIRED

AT27. Threat name: An attacker can perform UI redressing attacks [T-USER-INTERFACE-REDRESSING-ATTACK]

- **State:** Accepted
- **Description:**

General Threat Description:
UI redressing is a type of attack where an attacker manipulates the visual presentation of an application's user interface to deceive the user into performing unintended actions. This attack is closely related to clickjacking but can involve additional elements such as misleading visual cues, overlaying deceptive content, or rearranging UI components to induce errors. The goal is often to obtain sensitive information, trigger unauthorized commands, or mislead users into interacting with malicious elements.

Threat Agents/Attack Vectors:
UI redressing can be executed by external attackers who may craft malicious web pages or applications that load the target UI elements in a deceptive manner. Common attack vectors include:
Embedding the legitimate application within a custom, deceptive frame or overlay.
Manipulating CSS and HTML to alter the appearance or layout of legitimate UI components.
Using floating elements or other graphical tricks to cover or modify the appearance of the real interface.

Impacts:
The impacts of UI redressing attacks include:
Unauthorized transactions or operations performed without the user's genuine consent.
Compromise of sensitive information, such as login credentials or personal data.
Damage to the user's trust in the application's integrity and security, potentially affecting the reputation and reliability of the service provider.

Example Attack Scenarios:
Scenario 1: An attacker designs a webpage that looks like a popular video streaming service's login page. The attacker's page is actually an overlay on the legitimate service. When users enter their credentials, they believe they are logging into the streaming service, but their username and password are sent to the attacker.
Scenario 2: A user visits a custom-crafted website that appears to offer a benign survey. However, the survey's "Submit" button is strategically placed over a digital wallet's "Send Money" button, which is loaded invisibly in the background via an iframe. As the user attempts to submit the survey, they unintentionally authorize a payment to an account controlled by the attacker.
These scenarios exploit the visual trust users have in familiar interfaces, turning ordinary interactions into harmful actions without the user's knowledge.

- **C37. Countermeasure name:** Use multi-factor authentication
  - **State:** REQUIRED
- **C38. Countermeasure name:** Implement visual cues and indicators
  - **State:** REQUIRED

AT28. Threat name: An attacker can perform cross-site scripting (XSS) attacks [T-USER-INTERFACE-XSS-ATTACK]

- **State:** Accepted
- **Description:**

General Threat Description:
Cross-Site Scripting (XSS) attacks occur when an attacker injects malicious scripts into web pages viewed by other users. These scripts can execute in the context of the user's session, allowing the attacker to bypass normal access controls and directly manipulate the user interface or steal sensitive information such as session tokens, cookies, or personal data. XSS attacks typically exploit vulnerabilities in web applications that fail to properly sanitize input data before including it in output.

Threat Agents/Attack Vectors:
Threat agents include anyone with the ability to manipulate input fields where script injection is possible. Attack vectors are typically through:
User inputs that are improperly sanitized, allowing script injection into dynamically generated web pages.
Malicious payloads delivered via URL parameters, form fields, or even via manipulated documents that the user interacts with, such as PDFs or emails.

Impacts:
The impacts of XSS attacks can be severe, including:
Session hijacking, where attackers gain control of a user's active session.
Defacement of the website, which can damage the organization's reputation.
Phishing attacks orchestrated through the trusted site by displaying deceptive content.
Installation of malware or initiating unauthorized transactions or actions on behalf of the user.

Example Attack Scenarios:
Scenario 1: An attacker places a malicious script in a comment on a public forum that lacks proper input sanitization. When other users view the comment, the script executes within their browser, stealing their session cookies and sending them back to the attacker. The attacker then uses these cookies to hijack the users' sessions and gain unauthorized access to their accounts.
Scenario 2: A user receives an email with a link that appears to lead to a legitimate survey hosted by their bank. The link actually redirects to a version of the bank's login page that includes an XSS payload in the URL. When the user logs in, the script executes and captures their keystrokes, sending their username and password to the attacker.
In both scenarios, the attacker exploits the trust users have in the application's security, using the application itself as a vector to deliver harmful scripts.

- **C39. Countermeasure name:** Implement input validation and sanitization
  - **State:** REQUIRED
- **C40. Countermeasure name:** Use content security policy (CSP)
  - **State:** REQUIRED

Component: WebView

AT29. Threat name: WebView injection attacks [T-WEBVIEW-04]

- **State:** Accepted
- **Description:**

General threat description: WebView injection attacks occur when an attacker injects malicious code into the WebView. This can happen through untrusted web content or by exploiting vulnerabilities in the WebView component itself.
Threat agents/Attack vectors: Attackers can use various vectors such as malicious websites, compromised content delivery networks (CDNs), or man-in-the-middle attacks to inject malicious code into the WebView.
Impacts: Successful WebView injection can lead to unauthorized actions being performed within the application, data leakage, or the installation of malicious software.
Example Attack Scenarios: An attacker compromises a CDN serving JavaScript libraries to a web page loaded in WebView, injecting code that steals user data or performs unauthorized actions.

- **C41. Countermeasure name:** Enforce code integrity and content security policies
  - **State:** REQUIRED

AT30. Threat name: Man-in-the-middle (MitM) attack on WebView [T-WEBVIEW-02]

- **State:** Accepted
- **Description:**

  General threat description: A Man-in-the-Middle (MitM) attack occurs when an attacker intercepts and possibly alters the communication between two parties without their knowledge. In the context of WebView, if the web content is loaded over an insecure HTTP connection, it can be intercepted and modified by an attacker.
  Threat agents/Attack vectors: Attackers can exploit public Wi-Fi networks or compromised routers to intercept and modify HTTP traffic between the WebView and the web server.
  Impacts: MitM attacks can lead to sensitive data being disclosed, altered web content, and the injection of malicious scripts.
  Example Attack Scenarios: An attacker on a public Wi-Fi network intercepts HTTP traffic from the WebView, injecting malicious JavaScript to steal user credentials.

- **C42. Countermeasure name:** Enforce HTTPS and certificate pinning
  - **State:** REQUIRED

AT31. Threat name: Sensitive data is exposed through WebView [T-WEBVIEW-05]

- **State:** Accepted
- **Description:**

  General threat description: Sensitive data exposure occurs when confidential or private information is accessible to unauthorized parties. In the context of WebView, this can happen if sensitive data is stored or transmitted insecurely.
  Threat agents/Attack vectors: Attackers can exploit insecure storage, transmission vulnerabilities, or web vulnerabilities to access sensitive data displayed in or transmitted through WebView.
  Impacts: Exposure of sensitive data can lead to identity theft, financial loss, and reputational damage for both users and the organization.
  Example Attack Scenarios: A WebView displaying user account details without proper encryption can be intercepted by an attacker, leading to data theft.

- **C43. Countermeasure name:** Secure data storage and transmission
  - **State:** REQUIRED

AT32. Threat name: Untrusted content loading in WebView [T-WEBVIEW-03]

- **State:** Accepted
- **Description:**

  General threat description: Loading untrusted or malicious content in a WebView can lead to significant security risks. Untrusted content may include phishing pages, malware, or sites designed to exploit vulnerabilities in the WebView component.
  Threat agents/Attack vectors: Attackers can trick users into loading malicious URLs in WebView through phishing emails, malicious ads, or social engineering techniques.
  Impacts: This threat can lead to data theft, malware infection, or unauthorized actions being performed in the context of the WebView.
  Example Attack Scenarios: A user clicks on a link in a phishing email, which opens a malicious page in WebView that mimics a legitimate login page to steal credentials.

- **C44. Countermeasure name:** Enforce content filtering and URL whitelisting
  - **State:** REQUIRED

AT33. Threat name: Cross-site scripting (XSS) via WebView [T-WEBVIEW-01]

- **State:** Accepted
- **Description:**

  General threat description: Cross-Site Scripting (XSS) is an attack where malicious scripts are injected into otherwise benign and trusted web content. When a WebView displays web content, it may unintentionally execute malicious JavaScript if the content is not properly sanitized.
  Threat agents/Attack vectors: Attackers can exploit WebView by injecting malicious scripts through user input fields, URLs, or external web content loaded into the WebView.
  Impacts: XSS can lead to data theft, session hijacking, or spreading of malware. It compromises the confidentiality and integrity of the application.
  Example Attack Scenarios: An attacker crafts a malicious link that, when clicked by a user, loads a web page with injected JavaScript into the WebView, stealing the user's session cookies.

- **C45. Countermeasure name:** Input validation and output encoding
  - **State:** REQUIRED

End of Technical Threat report